

Foundations of RMSprop optimizer

[RMSprop optimizer]

1.0 Content Block

$$w^{\text{new}} = w^{\text{old}} + \eta \frac{1}{\sqrt{1 + \eta \sum_{i=1}^n (y_i - x_i' w^{\text{old}})^2}} (y_i - x_i' w^{\text{old}}) x_i.$$

2.0 Content Block

subject to additional stochastic noise. This approximation is only valid on a finite time-horizon in the following sense: assume that all the coefficients Q_i are sufficiently smooth. Let $T > 0$ and $g : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be a sufficiently smooth test function. Then, there exists a constant $C > 0$ such that for all $\eta > 0$

3.0 Content Block

where, γ is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

4.0 Content Block

Linear regression Suppose we want to fit a straight line $y = w_1 + w_2 x$ to a training set with observations $((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$ and corresponding estimated responses $(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$ using least squares. The objective function to be minimized is

5.0 Content Block

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \leftarrow \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial}{\partial w_1} (w_1 + w_2 x_i - y_i)^2 \\ \frac{\partial}{\partial w_2} (w_1 + w_2 x_i - y_i)^2 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} 2(w_1 + w_2 x_i - y_i) x_i \\ 2(w_1 + w_2 x_i - y_i) \end{bmatrix}.$$

Note that in each iteration or update step, the gradient is only evaluated at a single x_i . This is the key difference between stochastic gradient descent and batched gradient descent. In general, given a linear regression $y = \sum_{k \in 1:m} w_k x_k$ problem, stochastic gradient descent behaves differently when $m < n$ (underparameterized) and $m \geq n$ (overparameterized). In the overparameterized case, stochastic gradient descent converges to $\arg \min_{w: w^T x_k = y_k \forall k \in 1:n} \|w - w_0\|$. That is, SGD converges to the interpolation solution with minimum distance from the starting w_0 . This is true even when the learning rate remains constant. In the underparameterized case, SGD does not converge if learning rate remains constant.

6.0 Content Block

for $\Sigma(w) = \frac{1}{n^2} \left(\sum_{i=1}^n Q_i(w) - Q(w) \right) \left(\sum_{i=1}^n Q_i(w) - Q(w) \right)^T$

where $\int_t^T \sigma_s dB_s$ denotes the Ito-integral with respect to a Brownian motion is a more precise approximation in the sense that there exists a constant $C > 0$ such that

7.0 Content Block

History In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods, preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation") was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

8.0 Content Block

$$v_w(t) := \beta^2 v_w(t-1) + (1 - \beta^2) (\nabla_w L(t-1))^2 \quad \{displaystyle v_w(t) := \beta^2 v_w(t-1) + (1 - \beta^2) (\nabla_w L(t-1))^2\}$$

9.0 Content Block

$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$. $\{displaystyle \bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i\}$ When optimization is done, this averaged parameter vector takes the place of w .

10.0 Content Block

where the parameter w $\{displaystyle w\}$ which minimizes $Q(w)$ $\{displaystyle Q(w)\}$ is to be estimated, η $\{displaystyle \eta\}$ is a step size (sometimes called the learning rate in machine learning) and α $\{displaystyle \alpha\}$ is an exponential decay factor between 0 and 1 that determines the relative contribution of the current gradient and earlier gradients to the weight change. The name momentum stems from an analogy to momentum in physics: the weight vector w $\{displaystyle w\}$, thought of as a particle traveling through parameter space, incurs acceleration from the gradient of the loss ("force"). Unlike in classical stochastic gradient descent, it tends to keep traveling in the same direction, preventing oscillations. Momentum has been used successfully by computer scientists in the training of artificial neural networks for several decades. The momentum method is closely related to underdamped Langevin dynamics, and may be combined with simulated annealing. In mid-1980s the method was modified by Yurii Nesterov to use the gradient predicted at the next point, and the resulting so-called Nesterov Accelerated Gradient was sometimes used in ML in the 2010s.